
SelfJudge: Faster Speculative Decoding via Self-Supervised Judge Verification

Kanghoon Yoon^{† 1} Minsub Kim² Sungjae Lee² Joonhyung Lee² Sunghyeon Woo² Yeonjun In¹
Se Jung Kwon^{§ 2} Chanyoung Park¹ Dongsoo Lee^{§ 2}

Abstract

Speculative decoding accelerates LLM inference by verifying candidate tokens from a draft model against a larger target model. Recent “judge” decoding boosts this process by relaxing verification criteria by accepting draft tokens that may exhibit minor discrepancies from target model output, but existing methods are restricted by their reliance on human annotations or tasks with verifiable ground truths, limiting generalizability across diverse NLP tasks. We propose *SelfJudge*, which trains judge verifiers via self-supervision of the target model. Our method measures semantic preservation by assessing whether token-substituted responses preserve the meaning of original responses, enabling automatic verifier training across diverse NLP tasks. Our experiments show *SelfJudge* achieves superior inference-accuracy trade-offs than judge decoding baselines, offering a broadly applicable solution for faster LLM inference.

1. Introduction

Modern large language models (LLMs) have made significant strides in natural language processing, demonstrating competitive performance across a wide range of tasks (Radford et al., 2019; OpenAI et al., 2024). Empirical scaling laws establish a relationship between the number of parameters and model capability, as evidenced by models with hundreds of billions of parameters achieving state-of-the-art results on benchmarks (Kaplan et al., 2020; Grattafiori et al., 2024). However, the autoregressive generation process requires accessing all model parameters for each forward pass, creating a memory bandwidth bottleneck that impacts token generation latency. Furthermore, current trends toward more sophisticated LLM applications, such as multi-hop

reasoning (Wei et al., 2022), tool integration (Patil et al., 2024), and reasoning capability (Yang et al., 2025; DeepMind, 2025), produce longer output sequences, amplifying the computational burden of inference.

One prominent approach to address inference latency is Speculative Decoding (SD), which achieves partial parallelization of the generation process (Leviathan et al., 2023; Chen et al., 2023). Standard SD operates by deploying a computationally efficient *draft* model to propose candidate token sequences, which are subsequently validated in parallel by the *target* model (the model of interest). The acceptance criterion for draft tokens relies on a probability-based alignment verification: draft tokens are accepted when their likelihood under the target model meets or exceeds their likelihood under the draft model. This probabilistic acceptance mechanism guarantees distributional equivalence with standalone target model generation, preserving output quality while achieving computational speedup (Li et al., 2024a; Sun et al., 2023; 2024; Hu et al., 2025; Zhou et al., 2024).

Recently, Bachmann et al. (2025) and Garipov et al. (2025) revealed that alignment-based verification approaches are overly conservative and lack contextual awareness. Specifically, when minor lexical variations occur between draft and target models without altering the contextual meaning, conventional SD rejects these draft tokens. The conservative behavior limits the speedup potential of SD even when output quality remains uncompromised. To address the limitation, Bachmann et al. (2025) proposed the judge decoding that relaxes strict token alignment by evaluating semantic compatibility through a learnable *judge verifier*, enabling greater inference acceleration.

However, judge verification approaches face substantial practical limitations in obtaining judge verifier. Effective judge verifiers require token-level supervision data indicating acceptance decisions for individual tokens, a challenging annotation requirement. Recent approaches have attempted to address this through human annotation (Bachmann et al., 2025) or automated labeling based on task correctness (Garipov et al., 2025), but both strategies exhibit fundamental constraints. Human-annotated approaches introduce bias due to annotation subjectivity and divergence from target model decisions. In contrast, automated methods

[†]Work Done during an internship at NAVER Cloud. [§] Currently work at a2sys. ¹KAIST, Daejeon, South Korea ²NAVER Cloud, Seongnam, South Korea.

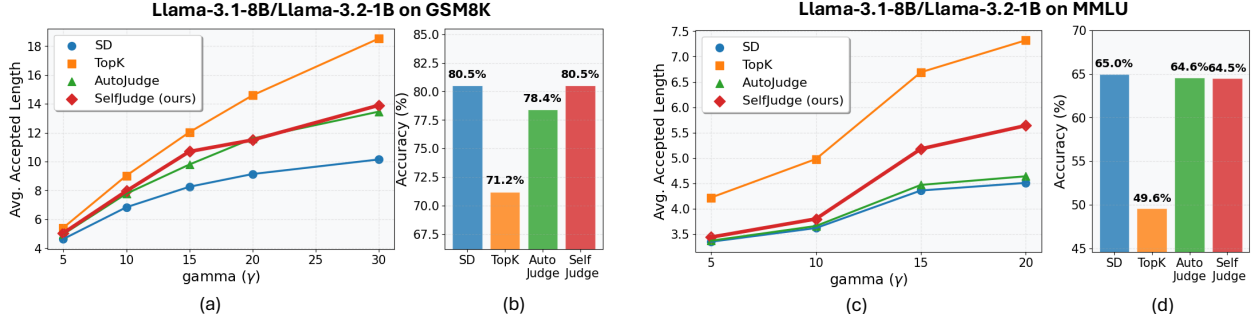


Figure 1. Inference efficiency and task performance comparison of SD methods on GSM8K (a,b) and MMLU (c,d). AutoJudge shows domain-specific limitations, performing well on mathematical reasoning but poorly on general knowledge tasks, while SelfJudge maintains consistent performance across both domains. γ represents the number of tokens generated by draft model per step.

are constrained to tasks with verifiable ground truth, restricting applicability to specific domains such as mathematical reasoning and code generation. This domain constraint is demonstrated empirically in Figure 1, where automated approaches such as AutoJudge achieve substantial speedup on mathematical reasoning tasks (GSM8K) but exhibit significant decrease on general knowledge evaluation (MMLU).

In this paper, we propose SelfJudge, a speculative decoding framework that enables judge verification across general NLP tasks without requiring human supervision or verifiable ground truth. Our approach exploits self-supervision from the target model to generate verifier training data. We introduce a criterion for token acceptability based on semantic preservation, quantified as the likelihood difference between the original target response and token-substituted variants under the target model’s distribution. Tokens whose replacement minimally affects the target model’s response likelihood are labeled as acceptable for training; tokens that substantially alter likelihood are rejected. This semantic preservation criterion enables automatic supervision generation across diverse task domains, including open-ended question answering and text summarization. The generated data is used to train a lightweight verifier that performs context-aware token evaluation during inference, accepting draft proposals based on semantic compatibility rather than distribution alignment.

We evaluate SelfJudge on diverse NLP tasks and datasets, including GSM8K, MATH-500, MMLU, CNN/DailyMail and LiveCodeBench. We demonstrate that SelfJudge outperforms existing verification methods by achieving superior accuracy-efficiency trade-offs across domains. While prior judge verification methods like AutoJudge achieve +1.96 accepted tokens per verification cycle with significant task performance degradation (-2.7% accuracy), our approach delivers enhanced +2.06 accepted tokens per verification cycle with only -1.0% accuracy. The improvement demonstrates SelfJudge’s superior applicability to general-purpose NLP scenarios, maintaining

task quality while enabling efficient speculative decoding across diverse domains.

In summary, we make the following contributions:

- We propose SelfJudge, a novel judge verification approach that enables generalization across diverse NLP tasks by accepting semantically coherent output tokens for faster LLM inference.
- We introduce an automatic training data generation method for the judge verifier that labels tokens using a semantic preservation score, measured by the target model’s likelihood differences.
- We conduct comprehensive experiments across NLP tasks using the Llama-3 and Qwen-2.5 families, demonstrating that SelfJudge generally outperforms existing verification methods in terms of token generation speed and task performance.

2. Related Works

2.1. Speculative Decoding

SD is a partial parallelization approach that addresses the latency of autoregressive generation. The core idea is to use an efficient module to generate drafts, which are then verified in parallel by the target model. Existing SD methods can be categorized into parameterized and parameter-free approaches. First, several SD methods have used small language models from the same family to suggest drafts (Leviathan et al., 2023; Miao et al., 2024). To enhance the alignment between draft and target models, advanced approaches propose specialized drafting architectures that utilize parts in the target model (e.g., hidden representation, LM head) (Stern et al., 2018; Cai et al., 2024; Li et al., 2024a; 2025). Moreover, recent approaches tried to improve the distribution alignment through knowledge distillation, which improves the draft acceptance rate (Zhou et al., 2024; Liu et al., 2024). In contrast, parameter-free SD methods

produce a draft using tokens from user prompts or generated history (Fu et al., 2024; Luo et al., 2025; Oliaro et al., 2025). While previous approaches have focused on improving the drafting phase to generate draft tokens that the target model is likely to accept, our work focuses on developing an improved verification method that allows the acceptance of these tokens without altering their semantic meaning. In this paper, our goal is to achieve speedup while maintaining task performance through our proposed verification approach.

2.2. Judge Speculative Decoding

Standard SD methods perform draft verification based on modified rejection sampling (Chen et al., 2023), which accepts tokens when the token probabilities from the target model meet or exceed those of the draft model. The advantage of using rejection sampling is that it preserves the distribution of the target model without degrading output quality. However, this alignment-based verification is overly conservative, which limits the speedup of SD when there are minor token discrepancies between draft and target models. To address this limitation, JudgeDecoding (Bachmann et al., 2025) have been proposed. JudgeDecoding utilizes a lightweight classifier as verifier, which examines the hidden representations of tokens. However, training such verifiers remains challenging as it requires labels indicating whether each token should be accepted or not. Hence, AutoJudge (Garipov et al., 2025) automates the data generation process by using verifiable ground truth in math and coding tasks. Specifically, AutoJudge adopts the answer-preservation approach, where it systematically tests token alternatives while maintaining correctness of math and coding problems. The method replaces individual tokens in a correct response and completes the modified response. If the final answer remains correct, it considers the replaced token as acceptable. Despite these advances, current judge verification methods lack generalization ability across diverse NLP tasks, showing performance improvements primarily on math and coding tasks. In this paper, we propose a judge verification approach that trains the verifier using semantic coherence from the original response, aiming to improve generalization across diverse NLP domains.

3. Method

In this section, we propose *SelfJudge*: a novel speculative decoding approach that enables judge verification across general NLP tasks without requiring human annotations or ground truth answers. The key innovation of *SelfJudge* lies in leveraging the semantic information inherent in the target model to automatically generate verifier training data.

As shown in Figure 2, *SelfJudge* introduces a self-data-generation paradigm where the target model assesses the

semantic change of the original response with a token substituted. Specifically, we define and measure semantic preservation by comparing the likelihood in the target model’s original response versus token-substituted variants. Tokens whose replacement does not significantly affect the model’s overall response confidence are labeled as acceptable, enabling automatic generation of high-quality training data across diverse NLP domains. This semantic preservation-based approach fundamentally shifts the verification paradigm from exact token matching or task-specific correctness to general semantic preservation. By obtaining the semantic preservation in the perspective of the target model, our approach makes it applicable to any NLP tasks, including open-ended tasks (e.g., creative writing) where ground truth is unavailable.

3.1. Conventional Speculative Decoding

We begin by introducing conventional SD. The core idea of SD is to efficiently generate drafts and verify them in parallel. Specifically, we generate γ candidate tokens using a small language model (Leviathan et al., 2023) or a specialized autoregressive model (Li et al., 2024a) to create drafts. When these generated candidate tokens are fed into the target model, we can verify whether the tokens are sufficiently aligned with the target model’s output using the probability value of next token.

Draft Generation. Formally, let $\mathcal{M}_{\text{target}}$ be the large language model of interest, and $\mathcal{M}_{\text{draft}}$ be the efficient model with a smaller number of parameters compared to $\mathcal{M}_{\text{target}}$. In the drafting phase, given prefix $\mathbf{x}_{\leq i}$, we first generate γ tokens using the efficient model autoregressively:

$$d_{i+1}, q_{i+1} = \mathcal{M}_{\text{draft}}(\mathbf{x}_{\leq i}), \text{ where } i = t, \dots, t + \gamma - 1 \quad (1)$$

where d_{i+1} represents the generated draft token and q_{i+1} denotes its corresponding probability.

Alignment-based Verification. Subsequently, we feed the draft tokens into the target model to verify whether each token is valid from the target model’s perspective:

$$(p_{t+1}, \dots, p_{t+\gamma+1}) = \mathcal{M}_{\text{target}}([\mathbf{x}_{\leq t}; d_{t+1}, \dots, d_{t+\gamma}]) \quad (2)$$

where p_t represents the target model’s probability value for each draft token d_t . The last probability $p_{t+\gamma+1}$ corresponds to additionally sampled token using the target model.

Conventional SD typically employs rejection sampling to verify draft tokens by comparing probability distributions from both the draft and target models for each token, accepting tokens only when the target model probabilities are high or similar to the draft probabilities as follows:

$$\text{Accept } d_t \text{ if } u_t < r_t, \text{ otherwise reject } d_t \\ \text{where } r_t = \min\left(1, \frac{p_t}{q_t}\right), \quad u_t \sim \text{Unif}(0, 1) \quad (3)$$

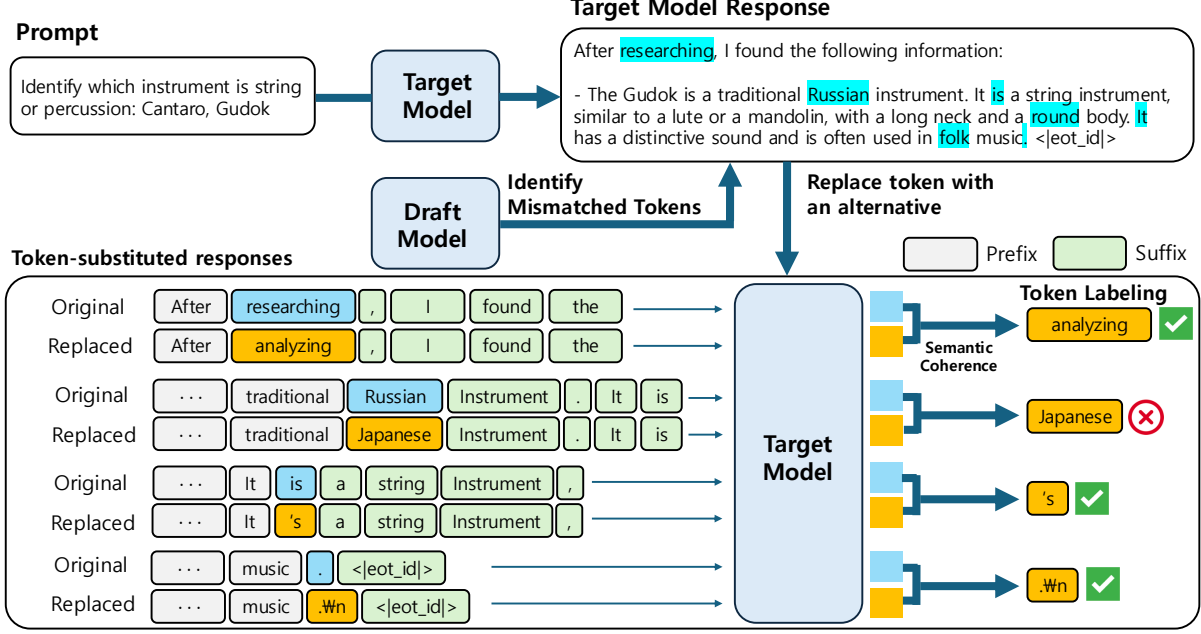


Figure 2. The training data generation process of SelfJudge. Our approach compares the likelihood of the replaced response with the original response to measure the semantic preservation score. If semantic preservation score is higher than τ , the replaced token is labeled as acceptable. After the token labeling process, we train the verifier that will be used during the inference phase for draft verification.

From the draft sequence, we accept tokens sequentially until the first rejection occurs according to Eqn.(3). It ensures that the accepted draft tokens maintain the target distribution. Beyond rejection sampling, other alignment-based methods exist, such as strict speculative sampling (He et al., 2024), which accepts draft tokens only when they match the target model’s greedy predictions exactly.

3.2. Judge Verification

As alignment-based verification methods have strict criteria, they lack context-awareness. This strict criterion can be overly conservative, for instance, rejecting “It’s a strong instrument” when the target model produces “It is a strong instrument”, even though both expressions preserve the same semantic meaning in context. Such unnecessary rejections limit the potential speedup of SD.

To address this issue, (Bachmann et al., 2025) introduces judge verification to complement alignment-based methods by recovering semantically acceptable tokens that would otherwise be conservatively rejected. Given draft tokens $d_{t+1}, \dots, d_{t+\gamma}$, we first obtain the hidden representations of the target model:

$$(h_{t+1}, \dots, h_{t+\gamma}) = \mathcal{M}_{\text{target}}([x_{\leq t}; d_{t+1}, \dots, d_{t+\gamma}]) \quad (4)$$

The judge verifier then evaluates each token based on its hidden representation:

$$\text{Verifier}(h_t) > \theta \implies \text{Accept } d_t \quad (5)$$

where Verifier is a learned binary classifier (we use logistic regression for simplicity), and θ is a hyperparameter determined via holdout validation. In practice, judge verification operates in parallel with alignment-based verification (e.g., rejection sampling). A token is accepted if *either* the judge or the alignment-based verifier approves it, ensuring that semantically coherent tokens are not unnecessarily discarded while maintaining the distributional guarantees of alignment-based verification.

3.3. Generate Data for Judge Verifier

For efficient and accurate judge decoding, obtaining a high-quality judge verifier is paramount. Our core insight is to leverage the target model’s own semantic understanding to train the verifier. By doing so, we increase the accepted tokens by allowing semantically valid tokens that may have a lower probability in the target model while maintaining response quality. Consequently, the judge verifier must identify which tokens preserve the original semantic meaning within the given context.

3.3.1. SEMANTICS PRESERVATION TOKEN LABELING

To maintain response quality in judge verification, previous methods have relied on human annotators (Bachmann et al., 2025) or on whether the final answers are preserved when tokens are replaced (Garipov et al., 2025). However, human annotations suffer from subjectivity, and the answer-

preservation-based method is limited to tasks with verifiable ground truths.

Rather than relying on external supervision, we utilize the target model’s self-evaluation. Since the goal of SD is to preserve the target model’s generation quality while improving latency, the target model itself is the best judge of semantic preservation. In our approach, the target model directly assesses the semantic impact of token replacements within its own responses. This enables automatic generation of high-quality training data across diverse NLP tasks without domain constraints.

Step 1. Identifying Mismatched Tokens. To improve the efficiency of SD with a verifier, we first choose which tokens to label, focusing on mismatched tokens where the target and draft models disagree. These tokens are candidates for relaxed verification, whereas matched tokens are already handled by the alignment-based verification phase of SD.

Formally, we generate response y using the target model given an instruction. We then feed response y to the draft model and obtain the draft model’s predictions at each position. We identify mismatched positions i where the target token y_i differs from the draft model’s top prediction $\arg \max P_{\text{draft}}(\cdot|y_{<i})$.

Step 2: Computing Semantic Preservation Score. When the draft model’s token does not match the token output by the target model, we need to determine if the alternative token from the draft model, i.e., $\arg \max P_{\text{draft}}(\cdot|y_{<i})$, is a semantically acceptable substitute to the target model. To this end, we introduce a *semantic preservation score*. Our core insight is to consider the alternative token as acceptable if it preserves the semantic meaning of the target model’s response. Therefore, our semantic preservation score quantifies how much of the semantic meaning of the target model’s response is preserved when the target model’s original token is replaced by the draft’s alternative.

Formally, given the target response y and the mismatched index i , we define the following *semantic preservation score* for token replacement z_i as:

$$s(y, z_i) = \log P(z_i|y_{<i}, y_{>i}) - \log P(y_i|y_{<i}, y_{>i}) \quad (6)$$

where z_i is the alternative token from the draft model and $y_{>i}$ represents the future token sequence. In practice, our semantic preservation score uses N future tokens as opposed to using all future tokens. This score measures how semantically similar the replacement token is to the target model’s established context. This bidirectional scoring is used solely for generating verifier training labels; during inference, the trained verifier operates on forward-pass without access to future tokens.

As many modern LLMs have adopted autoregressive inference, directly computing the probabilities conditioned on

the bidirectional context in Eqn.(6) is not possible. Therefore, we propose a method of transforming the bidirectional semantic difference into computationally tractable autoregressive operations as follows:

$$\begin{aligned} & \log P(z_i|y_{<i}, y_{>i}) - \log P(y_i|y_{<i}, y_{>i}) \\ &= \log P(z_i, y_{<i}, y_{>i}) - \log P(y_i, y_{<i}, y_{>i}) \\ &= \log \underbrace{P(y_{<i})}_{s_{\text{prefix}}} P(z_i|y_{<i}) P(y_{>i}|y_{<i}, z_i) \\ &\quad - \log \underbrace{P(y_{<i})}_{s_{\text{prefix}}} P(y_i|y_{<i}) P(y_{>i}|y_{<i}, y_i) \quad (7) \\ &= \underbrace{\log P(z_i|y_{<i}) - \log P(y_i|y_{<i})}_{s_{\text{prefix}}} \\ &\quad + \underbrace{\log P(y_{>i}|y_{<i}, z_i) - \log P(y_{>i}|y_{<i}, y_i)}_{\text{suffix likelihood}} \end{aligned}$$

s_{prefix} represents the relative probability given the prefix, computing the difference between the original and replaced token probabilities given the same prefix. The second term represents future context consistency, which computes the likelihood difference of future tokens in the target model’s response. Note that we obtain the probabilities conditioned on the replaced token by directly feeding the token-substituted response, i.e., $[y_{<i}, z_i, y_{>i}]$, into the target model.

Step 3: Token Labeling. Based on the semantic preservation score computed on the mismatched tokens, we finally obtain the label for each token. Specifically, a replaced token is labeled as acceptable if $s(y, z_i) > \tau$. Otherwise, the replaced token is labeled unacceptable, which implies that this replaced token should be rejected if it appears during the inference phase. For data construction, we save the pairs of hidden representations and labels (h_z, label) .

Our token label incorporates information about how closely the replaced token preserves the semantic information in the original response. We argue that SelfJudge is a more robust verification approach than the answer preservation-based approach, as our approach focuses on keeping the local semantics of the responses generated by the target model. Moreover, SelfJudge eliminates human bias when assessing the acceptable tokens.

Discussion on the Semantic Preservation Score. A straightforward approach to accepting the token from the draft model (accepting the replaced token z) is to compare the confidence of y and z . Some prior works (Kim et al., 2023; Narasimhan et al., 2025) have compared the probabilities solely on the prefix context, i.e., s_{prefix} in Eqn.(7), to accept high-confidence draft tokens.

Our bidirectional context approach enhances the criterion by incorporating the suffix from the original response, which reduces the uncertainty of replacement decisions. We formally define our semantic preservation score, $s(y, z_i)$, which can be decomposed to reveal its direct relationship with the

prefix-only baseline. Through a Bayesian interpretation, our score can be expressed as:

$$s(y, z_i) = s_{\text{prefix}}(y, z_i) + \underbrace{\log \left(\frac{P(y_{>i} | z_i, y_{<i})}{P(y_{>i} | y_i, y_{<i})} \right)}_{\text{Log Bayes Factor}} \quad (8)$$

This formulation demonstrates that our score begins with the baseline prefix score and refines it with an additional term, where the Log Bayes Factor quantifies the strength of evidence provided by the suffix. If the Log Bayes Factor is large, it indicates that the suffix is more probable given the replacement token, thus increasing the score. Conversely, if it is small, the suffix provides evidence for the original token, penalizing the replacement score. This transforms the decision-making process from a simple probability comparison into a principled, evidence-based update. Since obtaining the future token probabilities during the inference phase is not possible, we use the learned verifier to predict the semantic preservation score online.

3.4. Verifier Training

After collecting a dataset with semantically-preserved tokens, we can train a verifier that identifies acceptable tokens during the inference phase. It is important to note that we train a *single verifier*, and apply it across various NLP tasks. We generate 69,432 token labels using 1,220 prompts in GSM8K train, LiveCodeBench and Dolly15k datasets. While the judge verifier could be any type of model, we primarily select simple linear models that utilize existing LLM hidden states as input features, following (Bachmann et al., 2025). Specifically, we employ basic logistic regression for recognizing semantic-preserved tokens in our approach. To prevent overfitting, we conduct a straightforward grid search across L2 regularization parameters.

4. Experiment

4.1. Experimental Settings

Evaluation Protocol. We comprehensively evaluate our proposed method by following the SD literature (Bachmann et al., 2025). We select Llama-3.1-8B/Llama-3.2-1B and Qwen-2.5-8B/Qwen-2.5-0.5B as the target and draft models, respectively. To measure inference efficiency, we report the average accepted length (m). However, relaxed verification methods can introduce the possibility of accepting incorrect tokens, which may lead to task performance degradation. Hence, we report task-specific performance metrics with the average accepted lengths. We conduct the experiments on five evaluation scenarios, including GSM8K (Cobbe et al., 2021), MATH-500 (Lightman et al., 2023), LiveCodeBench (Jain et al., 2025), CNN/DailyMail, and MMLU (Hendrycks et al., 2021). Note that we ap-

ply chain-of-thought prompting (Wang et al., 2025) while the original MMLU benchmark employs a multiple-choice prompt that encourages language models to directly answer among four choices. This modification allows us to simultaneously assess both QA task performance and inference efficiency, taking into account the trade-off between accuracy and the average accepted lengths. For more details of the experimental setup, refer to Appendix C.

Baseline. For baseline comparisons, we include standard SD (Leviathan et al., 2023) with rejection sampling, which preserves the target model’s task performance. We also include top-k verification, which accepts draft tokens if they are among the target model’s top-k tokens, as a naive relaxed verification approach. Both methods represent alignment-based verification approach, as they determine acceptance based on token-level matching with the target model’s distribution. We also compare against judge verification approaches, specifically AutoJudge, which extends JudgeDecoding. We use AutoJudge instead of JudgeDecoding directly because the training data and implementation details of JudgeDecoding are not publicly available.

We train two versions of the AutoJudge verifier: AutoJudge and AutoJudge₊. Specifically, AutoJudge generates token labels based on the importance of each token to the final answer, using the training split of the GSM8K and LiveCodeBench datasets. While AutoJudge can assess token importance by checking whether a token replacement preserves the correctness on math and coding problems, this approach has limited applicability to open-ended NLP tasks. To address this limitation, we implement AutoJudge₊ by combining an LLMs-as-Judges framework (Li et al., 2024b) with AutoJudge. Specifically, we incorporate the instruction tuning dataset, Dolly15K, which encompasses diverse categories such as summarization, creative writing, and question answering. We train the verifier to act as a judge, asking it to determine if the original response and the token-substituted response are semantically equivalent.

Implementation Details. To set the threshold τ for semantic preservation labeling of SelfJudge, we use 100 sampled queries from GSM8K. In this math task, AutoJudge has identified ground-truth important tokens (i.e., tokens whose replacement changes the final answer). Specifically, we set $\tau = \text{quantile}(\text{semantic preservation scores of unacceptable tokens determined by AutoJudge}, 0.1)$ to ensure that all tokens rejected by AutoJudge are also rejected by SelfJudge. This conservative threshold guarantees high recall for answer-critical tokens of our method. Based on the established threshold τ , SelfJudge generates token labels for full training data. The suffix length for computing semantic preservation score is set to 20. For reproducibility, we set the temperature of the target model to 0 in all experiments.

Table 1. Performance comparison with SD baseline using Llama-3.1-8B/Llama-3.2-1B. **Green** = best performance gain compared to SD, **Yellow** = second place, **Red** = worst two. Δ_m : Average change in accepted length from SD, Δ_{task} : Average change in task performance from SD.

Method	GSM8K	MATH-500	MMLU	CNN/DM	Avg. Change Δ_m / Δ_{task}
	$m / \text{Acc.}$	$m / \text{Acc.}$	$m / \text{Acc.}$	m / FC	
SD	9.14 / 80.7	9.98 / 45.4	4.36 / 65.0	4.35 / 64.6	- / -
Top- k	14.59 / 71.2 (-9.5)	15.08 / 29.8 (-15.6)	6.69 / 49.5 (-15.5)	7.49 / 62.5 (-2.1)	+4.01 / -10.7
AutoJudge-R	9.71 / 80.1 (-0.6)	11.23 / 41.0 (-4.4)	4.47 / 64.6 (-0.4)	4.36 / 64.5 (-0.1)	+0.49 / -1.4
AutoJudge-F	11.63 / 78.4 (-2.3)	13.12 / 40.6 (-4.8)	5.15 / 63.1 (-1.9)	4.74 / 64.5 (-0.1)	+1.70 / -2.3
SelfJudge-R	10.09 / 80.7 (+0.0)	10.91 / 43.0 (-2.4)	5.14 / 64.4 (-0.6)	4.92 / 65.5 (+0.8)	+0.81 / -0.5
SelfJudge-F	11.29 / 80.5 (-0.2)	12.78 / 42.2 (-3.2)	6.38 / 62.7 (-2.3)	6.05 / 63.9 (-0.7)	+2.17 / -1.6

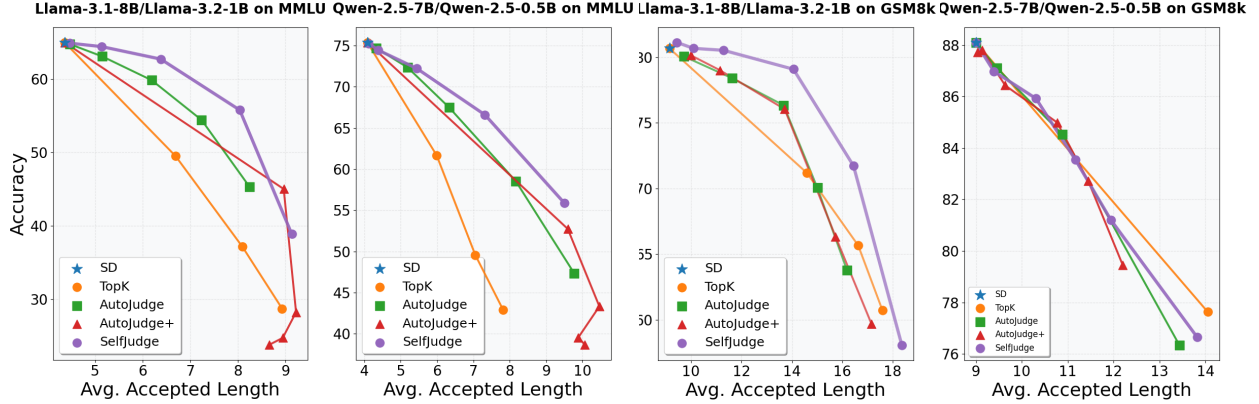


Figure 3. Speed/Performance comparison across different methods. We report the accuracy, with the corresponding average accepted length, by searching the threshold values of each method.

4.2. Performance Comparison Across NLP Tasks

We compare SelfJudge with baseline verification methods in a practical setting where we apply judge verification in diverse NLP scenarios. As judge verification methods have a trade-off between task performance and inference efficiency depending on the threshold value θ , we set a fixed threshold for the judge verifier using a holdout set in advance. {Method}-R and {Method}-F denote applying θ that achieved the best recall and best F1 score, respectively. In Table 1, we make the following observations:

Observation1. SelfJudge achieved significant speed-up over standard SD across various NLP tasks while demonstrating the smallest task performance degradation compared to SD. Particularly when we compare SelfJudge-F with AutoJudge-R, it achieves higher average accepted length while showing similar performance degradation across the GSM8K, MATH-500, MMLU, and CNN/DM datasets, resulting in greater inference speed improvements.

Observation2. SelfJudge has the advantage of consistently generalizing across all tasks. Specifically, existing baselines show significant performance loss on certain tasks. Top- k ($k = 2$) verification achieves large speed-ups but substantially degrades task performance. AutoJudge-R partially achieves meaningful speed-up without significant perfor-

mance drop compared to SD on GSM8K and MATH-500, but shows almost no speed-up on MMLU and CNN/DM. Furthermore, we observed that the judge verification of AutoJudge severely decreases Pass@1 on LiveCodeBench in Table 6 in Appendix. In contrast, both SelfJudge-R and SelfJudge-F consistently achieve speed-ups across all datasets with minimal performance degradation among all compared methods.

We argue that these results stem from the superiority of our semantic coherence-based token acceptance strategy. Specifically, SelfJudge is trained to accept tokens if they are semantically coherent in context, regardless of the task. In contrast, AutoJudge trains the verifier to accept tokens even when semantically different tokens appear in context, as long as the final answer remains identical to the original. This fundamental difference significantly impacts the robustness when the verifier encounters out-of-distribution tokens during actual inference scenarios.

4.3. Trade-off Analysis: Accepted Length vs. Task Performance

Since judge verification exhibits a trade-off between task performance and inference speed depending on the threshold setting, we performed additional analyzes using the

Table 2. Performance comparison on Llama-3.1-70B/8B. m denotes the average accepted length.

Method	GSM8K m / Acc	MMLU m / Acc
SD	9.532 / 93.0	5.668 / 80.6
SelfJudge-R ($\theta = 0.28$)	10.622 / 93.8	6.262 / 81.4
SelfJudge-F ($\theta = 0.38$)	11.663 / 94.0	6.955 / 79.8
SelfJudge ($\theta = 0.5$)	12.528 / 93.8	7.576 / 77.2

Llama-3.1-8B / Llama-3.2-1B and Qwen-2.5-7B / Qwen-2.5-0.5B models. Specifically, we varied the threshold θ of each judge verifier. From Figure 3, we make two key observations. First, SelfJudge shows competitive performance to AutoJudge on MMLU with CoT prompting, indicating that the token generation criteria of SelfJudge correctly labels the token compared to answer-preservation-based methods AutoJudge and AutoJudge+. These suggest that SelfJudge is consistently generalizing well in general reasoning tasks such as QA. Second, we observed that SelfJudge generally performs comparably to or better than AutoJudge methods on GSM8K. Although AutoJudge can determine the token label with the ground-truth answer for math/coding problems, our semantic-preserving approach remains competitive. This implies that determining token acceptance based on the semantic coherence in the local context is consistently better than judging token acceptance based on answer correctness, not only for general reasoning tasks but also for math and coding problems.

4.4. Performance on Larger Model

To examine the scalability of SelfJudge to larger models, we conducted additional experiments with Llama-3.1-70B/8B as target and draft models, respectively. Following the same protocol as our main experiments, we sampled 500 instances from GSM8K (train), 220 from Live Code Bench (train), and 500 from Dolly15k. SelfJudge generated a total of 53,318 token labels for verifier training. We exclude the result of AutoJudge since it could not complete the data generation within timeframes. Table 2 presents that SelfJudge maintains accuracy while improving the average of accepted length compared to SD, even on the 70B target model. This demonstrates that our approach scales effectively to larger models.

4.5. Wall Clock Time Evaluation

To demonstrate that SelfJudge achieves genuine wall-clock speedup beyond theoretical improvements, we conducted comprehensive throughput experiments by implementing our method in vLLM, the de facto standard framework for production-grade LLM serving. We measured end-to-end inference throughput (tokens per second) on

Table 3. Wall-clock throughput comparison on Llama-3.1-8B/Llama-3.2-1B using A100 GPU with 1-way tensor parallelism.

Dataset	Method	Tokens/sec	Acc (%)
GSM8K	SD	111.21	80.7
	AutoJudge-R	118.14	80.1
	SelfJudge-F	137.37	80.5
MMLU	SD	61.13	64.6
	AutoJudge-R	62.67	64.5
	SelfJudge-F	89.45	63.9

Table 4. Wall-clock throughput comparison on Llama-3.1-70B/8B using 4 A100 GPUs with 4-way tensor parallelism.

Dataset	Method	Tokens/sec	Acc (%)
GSM8K	SD	53.45	93.0
	SelfJudge-R	59.57	93.8
	SelfJudge-F	65.40	94.0
MMLU	SD	39.19	80.6
	SelfJudge-R	43.30	81.4
	SelfJudge-F	48.09	79.8

NVIDIA A100 GPUs, which represents the most widely adopted hardware configuration in both research and industrial deployments. We set the draft length γ to 20 for GSM8K and 15 for MMLU across all methods. For the experiment with Llama-3.1-8B/Llama-3.2-1B, we employed single-way tensor parallelism on a single A100 GPU. We use the 4-way tensor parallelism on 4 A100 GPUs for the experiment with Llama-3.1-70B/8B.

Results on Llama-3.1-8B/Llama-3.2-1B. Table 3 demonstrates that SelfJudge-F achieves substantially higher throughput than both standard speculative decoding (SD) and AutoJudge-R across both benchmark tasks. On GSM8K, SelfJudge-F delivers 23.5% improvement over SD and a 16.3% improvement over AutoJudge-R, while maintaining comparable accuracy. More remarkably, on MMLU, SelfJudge-F achieves a 46.3% speedup over SD and a 42.7% improvement over AutoJudge-R, with only a marginal accuracy difference.

Results on Llama-3.1-70B/8B. Table 4 presents the results for this larger model configuration. We skip reporting AutoJudge due to the expensive data generation cost. SelfJudge-R achieves 11.5% and 10.5% improvements over SD on GSM8K and MMLU, respectively. SelfJudge-F further pushes the throughput, achieving 22.4% and 22.7% speedups over SD. Importantly, these speedups are achieved while maintaining or even improving accuracy, confirming that SelfJudge scales effectively to larger models without sacrificing generation quality. These results confirm that the accepted length improvements we observed in prior experiments directly translate to real wall-clock speedup in

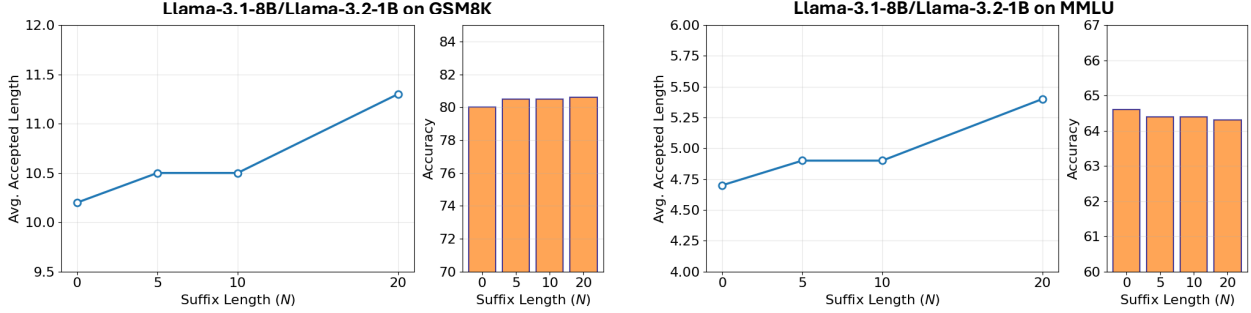


Figure 4. Performance of SelfJudge over a range of suffix length N . We compute the semantic score by including the likelihood computed on N future tokens.

production-grade serving systems.

Elapsed Time Analysis of Each Phase. Since our verification paradigm introduces an additional stage with the judge verifier, we measure the time elapsed for each stage on Llama-3.1-8B/Llama-3.2-1B with a single V100 GPU. Out of a total verification time of 7.515 sec, 5 draft token generation accounts for 4.51 sec, the target model forward pass for 2.81 sec, and rejection sampling for 0.175 sec, while the judge verifier inference takes only 0.02 sec (0.26% of the total). This confirms that our lightweight judge verifier incurs negligible verification cost, so any improvement in the accepted length of draft sequences directly translates into wall-clock-time gains for SD methods.

4.6. Semantic Preservation Score Analysis

When computing the semantic preservation score $s(y, z_i)$, we can adjust the length of the suffix $y_{>i}$. In Figure 4, we analyze the impact of the suffix length N on performance. We observe that the highest accuracy is achieved when the suffix length $N = 20$, while $N = 0$ yields the worst performance. This implies that considering bidirectional context is beneficial for measuring token semantic preservation, resulting in improved judge decoding performance. Moreover, it indicates that suffix tokens provide stronger evidence for semantic preservation than prefix-only approaches, validating our approach.

4.7. Qualitative Analysis

We analyze the most frequently accepted token transformations by SelfJudge across three distinct domains: mathematical reasoning, coding, and general reasoning. First, across all task domains, we observed that SelfJudge consistently relaxes draft verification by accepting tokens, which change the format (“ $\rightarrow$.”, “ $\backslash n \rightarrow \backslash n \backslash n$ ”) and equivalent sentence starters (“Let $\rightarrow$ To”, “Next $\rightarrow$ Now”). Second, we observed the domain-specific transformations. For example, we find mathematical term substitutions like “solve $\rightarrow$ find” and “x $\rightarrow$ *”, and coding style vari-

ations (“ $\backslash n \rightarrow \backslash n \backslash n$ ”) reflecting different programming styles with identical functionality. These patterns confirm that SelfJudge achieved great inference efficiency by accepting tokens with minor token discrepancy. Please refer to Appendix D.3 for more examples.

5. Conclusion

In this paper, we introduce a paradigm shift for judge verification in SD, moving beyond the conventional reliance on human annotations and task-specific ground truths. We present SelfJudge, a novel framework that pioneers a self-supervised approach, leveraging the target model itself to assess semantic preservation. Our method automatically constructs a verifier training data by quantifying the contextual impact of token substitutions through a principled “semantic preservation score” derived from the model’s own likelihoods. Empirical validation across a diverse suite of NLP benchmarks confirms that SelfJudge achieves substantial inference acceleration while maintaining high fidelity to the target model, establishing our method as a practical and scalable foundation for efficient large language model inference. Our current work adopts a simple linear verifier, and a broader exploration of more complex architectures remains an important next step. Consequently, we have not yet investigated the scaling laws that relate the volume of training data to verifier performance, an understanding of which will be crucial for improving the performance of SelfJudge.

Impact Statement

This research contributes to making large language model inference more efficient and accessible, which benefits society by reducing computational costs of AI deployment. By enabling faster inference while maintaining output quality, our approach helps democratize access to advanced language models across different socio-economic regions. We acknowledge that improved AI efficiency may have broader societal implications and commit to transparent reporting of our methods to enable responsible adoption. In compliance with ICML’s code of ethics, we have carefully considered potential negative consequences and the limitations of our approach. While we introduce some relaxation in draft verification that could lead to semantic drift, our extensive evaluation across diverse tasks demonstrates minimal performance degradation compared to standard speculative decoding.

Acknowledgement

This work was supported by NAVER Cloud and conducted at KAIST. This research was also supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (RS-2025-02304967, AI Star Fellowship(KAIST))

References

- Bachmann, G., Anagnostidis, S., Pumarola, A., Georgopoulos, M., Sanakoyeu, A., Du, Y., Schönfeld, E., Thabet, A., and Kohler, J. K. Judge decoding: Faster speculative sampling requires going beyond model alignment. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Forty-first International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=PEpbUobfJv>.
- Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling, 2023. URL <https://arxiv.org/abs/2302.01318>.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., , and Schulman, J. Measuring massive multitask language understanding, 2021. URL [arXiv:2110.14168](https://arxiv.org/abs/2110.14168), 2021.
- DeepMind, G. Gemini2.5: Our newest gemini model with thinking., 2025. URL <https://blog.google/technology/google-deepmind>.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. BERT: Pre-training of deep bidirectional transformers for language understanding. In Burstein, J., Doran, C., and Solorio, T. (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1423. URL <https://aclanthology.org/N19-1423/>.
- Fu, Y., Bailis, P., Stoica, I., and Zhang, H. Break the sequential dependency of llm inference using lookahead decoding, 2024. URL <https://arxiv.org/abs/2402.02057>.
- Garipov, R., Velikonitvsev, F., Svirschevski, R., Egiazarian, V., and Ryabinin, M. Autojudge: Judge decoding without manual annotation. In *Proceedings of the 39th International Conference on Neural Information Processing Systems*, 2025.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A., Korenev, A., Hinsvark, A., Rao, A., Zhang, A., Rodriguez, A., Gregerson, A., Spataru, A., Roziere, B., Biron, B., Tang, B., Chern, B., Caucheteux, C., Nayak, C., Bi, C., Marra, C., McConnell, C., Keller, C., Touret, C., Wu, C., Wong, C., Ferrer, C. C., Nikolaidis, C., Allonsius, D., Song, D., Pintz, D., Livshits, D., Wyatt, D., Esiobu, D., Choudhary, D., Mahajan, D., Garcia-Olano, D., Perino, D., Hupkes, D., Lakomkin, E., AlBadawy, E., Lobanova, E., Dinan, E., Smith, E. M., Radenovic, F., Guzmán, F., Zhang, F., Synnaeve, G., Lee, G., Anderson, G. L., Thattai, G., Nail, G., Mialon, G., Pang, G., Cucurell, G., Nguyen, H., Korevaar, H., Xu, H., Touvron, H., Zarov, I., Ibarra, I. A., Kloumann, I., Misra, I., Evtimov, I., Zhang, J., Copet, J., Lee, J., Geffert, J., Vranes, J., Park, J., Mahadeokar, J., Shah, J., van der Linde, J., Billoock, J., Hong, J., Lee, J., Fu, J., Chi, J., Huang, J., Liu, J., Wang, J., Yu, J., Bitton, J., Spisak, J., Park, J., Rocca, J., Johnston, J., Saxe, J., Jia, J., Alwala, K. V., Prasad, K., Upasani, K., Plawiak, K., Li, K., Heafield, K., Stone, K., El-Arini, K., Iyer, K., Malik, K., Chiu, K., Bhalla, K., Lakhota, K., Rantala-Yeary, L., van der Maaten, L., Chen, L., Tan, L., Jenkins, L., Martin, L., Madaan, L., Malo, L., Blecher, L., Landzaat, L., de Oliveira, L., Muzzi, M., Pasupuleti, M., Singh, M., Paluri, M., Kardas, M., Tsimpoukelli, M., Oldham, M., Rita, M., Pavlova, M., Kambadur, M., Lewis, M., Si, M., Singh, M. K., Hassan, M., Goyal, N., Torabi, N., Bashlykov, N., Bogoychev, N., Chatterji, N., Zhang, N.,

- Duchenne, O., Çelebi, O., Alrassy, P., Zhang, P., Li, P., Vasic, P., Weng, P., Bhargava, P., Dubal, P., Krishnan, P., Koura, P. S., Xu, P., He, Q., Dong, Q., Srinivasan, R., Ganapathy, R., Calderer, R., Cabral, R. S., Stojnic, R., Raileanu, R., Maheswari, R., Girdhar, R., Patel, R., Sauvestre, R., Polidoro, R., Sumbaly, R., Taylor, R., Silva, R., Hou, R., Wang, R., Hosseini, S., Chennabasappa, S., Singh, S., Bell, S., Kim, S. S., Edunov, S., Nie, S., Narang, S., Raparthy, S., Shen, S., Wan, S., Bhosale, S., Zhang, S., Vandenhende, S., Batra, S., Whitman, S., Sootla, S., Collot, S., Gururangan, S., Borodinsky, S., Herman, T., Fowler, T., Sheasha, T., Georgiou, T., Scialom, T., Speckbacher, T., Mihaylov, T., Xiao, T., Karn, U., Goswami, V., Gupta, V., Ramanathan, V., Kerkez, V., Gonguet, V., Do, V., Vogeti, V., Albiero, V., Petrovic, V., Chu, W., Xiong, W., Fu, W., Meers, W., Martinet, X., Wang, X., Wang, X., Tan, X. E., Xia, X., Xie, X., Jia, X., Wang, X., Goldschlag, Y., Gaur, Y., Babaei, Y., Wen, Y., Song, Y., Zhang, Y., Li, Y., Mao, Y., Coudert, Z. D., Yan, Z., Chen, Z., Papakipos, Z., Singh, A., Srivastava, A., Jain, A., Kelsey, A., Shajnfeld, A., Gangidi, A., Victoria, A., Goldstand, A., Menon, A., Sharma, A., Boesenberg, A., Baevski, A., Feinstein, A., Kallet, A., Sangani, A., Teo, A., Yunus, A., Lupu, A., Alvarado, A., Caples, A., Gu, A., Ho, A., Poulton, A., Ryan, A., Ramchandani, A., Dong, A., Franco, A., Goyal, A., Saraf, A., Chowdhury, A., Gabriel, A., Bharambe, A., Eisenman, A., Yazdan, A., James, B., Maurer, B., Leonhardi, B., Huang, B., Loyd, B., Paola, B. D., Paranjape, B., Liu, B., Wu, B., Ni, B., Hancock, B., Wasti, B., Spence, B., Stojkovic, B., Gamido, B., Montalvo, B., Parker, C., Burton, C., Mejia, C., Liu, C., Wang, C., Kim, C., Zhou, C., Hu, C., Chu, C.-H., Cai, C., Tindal, C., Feichtenhofer, C., Gao, C., Civin, D., Beaty, D., Kreymer, D., Li, D., Adkins, D., Xu, D., Testuggine, D., David, D., Parikh, D., Liskovich, D., Foss, D., Wang, D., Le, D., Holland, D., Dowling, E., Jamil, E., Montgomery, E., Presani, E., Hahn, E., Wood, E., Le, E.-T., Brinkman, E., Arcaute, E., Dunbar, E., Smothers, E., Sun, F., Kreuk, F., Tian, F., Kokkinos, F., Ozgenel, F., Caggioni, F., Kanayet, F., Seide, F., Florez, G. M., Schwarz, G., Badeer, G., Swee, G., Halpern, G., Herman, G., Sizov, G., Guangyi, Zhang, Lakshminarayanan, G., Inan, H., Shojanazeri, H., Zou, H., Wang, H., Zha, H., Habeeb, H., Rudolph, H., Suk, H., Aspegren, H., Goldman, H., Zhan, H., Damla, I., Molybog, I., Tufanov, I., Leontiadis, I., Veliche, I.-E., Gat, I., Weissman, J., Geboski, J., Kohli, J., Lam, J., Asher, J., Gaya, J.-B., Marcus, J., Tang, J., Chan, J., Zhen, J., Reizenstein, J., Teboul, J., Zhong, J., Jin, J., Yang, J., Cummings, J., Carvill, J., Shepard, J., McPhie, J., Torres, J., Ginsburg, J., Wang, J., Wu, K., U. K. H., Saxena, K., Khandelwal, K., Zand, K., Matosich, K., Veeraraghavan, K., Michelena, K., Li, K., Jagadeesh, K., Huang, K., Chawla, K., Huang, K., Chen, L., Garg, L., A. L., Silva, L., Bell, L., Zhang, L., Guo, L., Yu, L., Moshkovich, L., Wehrstedt, L., Khabsa, M., Avalani, M., Bhatt, M., Mankus, M., Hasson, M., Lennie, M., Reso, M., Groshev, M., Naumov, M., Lathi, M., Keneally, M., Liu, M., Seltzer, M. L., Valko, M., Restrepo, M., Patel, M., Vyatskov, M., Samvelyan, M., Clark, M., Macey, M., Wang, M., Hermoso, M. J., Metanat, M., Rastegari, M., Bansal, M., Santhanam, N., Parks, N., White, N., Bawa, N., Singhal, N., Egebo, N., Usunier, N., Mehta, N., Laptev, N. P., Dong, N., Cheng, N., Chernoguz, O., Hart, O., Salpekar, O., Kalinli, O., Kent, P., Parekh, P., Saab, P., Balaji, P., Rittner, P., Bontrager, P., Roux, P., Dollar, P., Zvyagina, P., Ratanchandani, P., Yuvraj, P., Liang, Q., Alao, R., Rodriguez, R., Ayub, R., Murthy, R., Nayani, R., Mitra, R., Parthasarathy, R., Li, R., Hogan, R., Battey, R., Wang, R., Howes, R., Rinott, R., Mehta, S., Siby, S., Bondu, S. J., Datta, S., Chugh, S., Hunt, S., Dhillon, S., Sidorov, S., Pan, S., Mahajan, S., Verma, S., Yamamoto, S., Ramaswamy, S., Lindsay, S., Lindsay, S., Feng, S., Lin, S., Zha, S. C., Patil, S., Shankar, S., Zhang, S., Zhang, S., Wang, S., Agarwal, S., Sajuyigbe, S., Chintala, S., Max, S., Chen, S., Kehoe, S., Satterfield, S., Govindaprasad, S., Gupta, S., Deng, S., Cho, S., Virk, S., Subramanian, S., Choudhury, S., Goldman, S., Remez, T., Glaser, T., Best, T., Koehler, T., Robinson, T., Li, T., Zhang, T., Matthews, T., Chou, T., Shaked, T., Vontimitta, V., Ajayi, V., Montanez, V., Mohan, V., Kumar, V. S., Mangla, V., Ionescu, V., Poenaru, V., Mihailescu, V. T., Ivanov, V., Li, W., Wang, W., Jiang, W., Bouaziz, W., Constable, W., Tang, X., Wu, X., Wang, X., Wu, X., Gao, X., Kleinman, Y., Chen, Y., Hu, Y., Jia, Y., Qi, Y., Li, Y., Zhang, Y., Zhang, Y., Adi, Y., Nam, Y., Yu, Wang, Zhao, Y., Hao, Y., Qian, Y., Li, Y., He, Y., Rait, Z., DeVito, Z., Rosnbrick, Z., Wen, Z., Yang, Z., Zhao, Z., and Ma, Z. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- He, Z., Zhong, Z., Cai, T., Lee, J., and He, D. REST: Retrieval-based speculative decoding. In Duh, K., Gomez, H., and Bethard, S. (eds.), *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 1582–1595, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.88. URL <https://aclanthology.org/2024.naacl-long.88/>.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding, 2021.
- Hu, Z., Zheng, T., Viswanathan, V., Chen, Z., Rossi, R. A., Wu, Y., Manocha, D., and Huang, H. Towards optimal multi-draft speculative decoding. In *ICLR*, 2025.
- Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T.,

- Wang, S., Solar-Lezama, A., Sen, K., and Stoica, I. Live-codebench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models, 2020. URL <https://arxiv.org/abs/2001.08361>.
- Kim, S., Mangalam, K., Moon, S., Malik, J., Mahoney, M. W., Gholami, A., and Keutzer, K. Speculative decoding with big little decoder. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS ’23, Red Hook, NY, USA, 2023. Curran Associates Inc.
- Laban, P., Schnabel, T., Bennett, P. N., and Hearst, M. A. SummaC: Re-visiting NLI-based models for inconsistency detection in summarization. *Transactions of the Association for Computational Linguistics*, 10:163–177, 2022. doi: 10.1162/tacl.a.00453. URL <https://aclanthology.org/2022.tacl-1.10/>.
- Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, ICML’23. JMLR.org, 2023.
- Li, Yuhui, Wei, Fangyun, Zhang, Chao, Zhang, and Hongyang. Eagle: speculative sampling requires rethinking feature uncertainty. In *International Conference on Macihne Learning*, ICML’24. JMLR.org, 2024a.
- Li, H., Dong, Q., Chen, J., Su, H., Zhou, Y., Ai, Q., Ye, Z., and Liu, Y. Llms-as-judges: A comprehensive survey on llm-based evaluation methods, 2024b. URL <https://arxiv.org/abs/2412.05579>.
- Li, Y., Wei, F., Zhang, C., and Zhang, H. Eagle-3: Scaling up inference acceleration of large language models via training-time test, 2025. URL <https://arxiv.org/abs/2503.01840>.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let’s verify step by step, 2023. URL <https://arxiv.org/abs/2305.20050>.
- Liu, X., Hu, L., Bailis, P., Cheung, A., Deng, Z., Stoica, I., and Zhang, H. Online speculative decoding. In *Proceedings of the 41st International Conference on Machine Learning*, ICML’24. JMLR.org, 2024.
- Luo, X., Wang, Y., Zhu, Q., Zhang, Z., Zhang, X., Yang, Q., and Xu, D. Turning trash into treasure: Accelerating inference of large language models with token recycling, 2025. URL <https://arxiv.org/abs/2408.08696>.
- Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Wang, Z., Zhang, Z., Wong, R. Y. Y., Zhu, A., Yang, L., Shi, X., Shi, C., Chen, Z., Arfeen, D., Abhyankar, R., and Jia, Z. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS ’24, pp. 932–949. ACM, April 2024. doi: 10.1145/3620666.3651335. URL <http://dx.doi.org/10.1145/3620666.3651335>.
- Nallapati, R., Zhou, B., dos Santos, C., Gulçehre, Ç., and Xiang, B. Abstractive text summarization using sequence-to-sequence RNNs and beyond. In *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning*. Association for Computational Linguistics, 2016. doi: 10.18653/v1/K16-1028.
- Narasimhan, H., Jitkrittum, W., Rawat, A. S., Kim, S., Gupta, N., Menon, A. K., and Kumar, S. Faster cascades via speculative decoding. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=vo9t20wsmd>.
- Oliaro, G., Jia, Z., Campos, D., and Qiao, A. Suffixdecoding: Extreme speculative decoding for emerging ai applications, 2025. URL <https://arxiv.org/abs/2411.04975>.
- OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., Avila, R., Babuschkin, I., Balaji, S., Balcom, V., Baltescu, P., Bao, H., Bavarian, M., Belgum, J., Bello, I., Berdine, J., Bernadett-Shapiro, G., Berner, C., Bogdonoff, L., Boiko, O., Boyd, M., Brakman, A.-L., Brockman, G., Brooks, T., Brundage, M., Button, K., Cai, T., Campbell, R., Cann, A., Carey, B., Carlson, C., Carmichael, R., Chan, B., Chang, C., Chantzis, F., Chen, D., Chen, S., Chen, R., Chen, J., Chen, M., Chess, B., Cho, C., Chu, C., Chung, H. W., Cummings, D., Currier, J., Dai, Y., Decareaux, C., Degry, T., Deutsch, N., Deville, D., Dhar, A., Dohan, D., Dowling, S., Dunning, S., Ecoffet, A., Eleti, A., Eloundou, T., Farhi, D., Fedus, L., Felix, N., Fishman, S. P., Forte, J., Fulford, I., Gao, L., Georges, E., Gibson, C., Goel, V., Gogineni, T., Goh, G., Gontijo-Lopes, R., Gordon, J., Grafstein, M., Gray, S., Greene, R., Gross, J., Gu, S. S., Guo, Y., Hallacy, C., Han, J., Harris, J., He, Y., Heaton, M., Heidecke, J., Hesse, C., Hickey, A., Hickey, W., Hoeschele, P., Houghton, B.,

- Hsu, K., Hu, S., Hu, X., Huizinga, J., Jain, S., Jain, S., Jang, J., Jiang, A., Jiang, R., Jin, H., Jin, D., Jomoto, S., Jonn, B., Jun, H., Kaftan, T., Łukasz Kaiser, Kamali, A., Kanitscheider, I., Keskar, N. S., Khan, T., Kilpatrick, L., Kim, J. W., Kim, C., Kim, Y., Kirchner, J. H., Kiros, J., Knight, M., Kokotajlo, D., Łukasz Kondraciuk, Kondrich, A., Konstantinidis, A., Kopic, K., Krueger, G., Kuo, V., Lampe, M., Lan, I., Lee, T., Leike, J., Leung, J., Levy, D., Li, C. M., Lim, R., Lin, M., Lin, S., Litwin, M., Lopez, T., Lowe, R., Lue, P., Makanju, A., Malfacini, K., Manning, S., Markov, T., Markovski, Y., Martin, B., Mayer, K., Mayne, A., McGrew, B., McKinney, S. M., McLeavey, C., McMillan, P., McNeil, J., Medina, D., Mehta, A., Menick, J., Metz, L., Mishchenko, A., Mishkin, P., Monaco, V., Morikawa, E., Mossing, D., Mu, T., Murati, M., Murk, O., Mély, D., Nair, A., Nakano, R., Nayak, R., Neelakantan, A., Ngo, R., Noh, H., Ouyang, L., O’Keefe, C., Pachocki, J., Paino, A., Palermo, J., Pantuliano, A., Parascandolo, G., Parish, J., Parparita, E., Passos, A., Pavlov, M., Peng, A., Perelman, A., de Avila Belbute Peres, F., Petrov, M., de Oliveira Pinto, H. P., Michael, Pokorný, Pokrass, M., Pong, V. H., Powell, T., Power, A., Power, B., Proehl, E., Puri, R., Radford, A., Rae, J., Ramesh, A., Raymond, C., Real, F., Rimbach, K., Ross, C., Rotsted, B., Roussez, H., Ryder, N., Saltarelli, M., Sanders, T., Santurkar, S., Sastry, G., Schmidt, H., Schnurr, D., Schulman, J., Selsam, D., Sheppard, K., Sherbakov, T., Shieh, J., Shoker, S., Shyam, P., Sidor, S., Sigler, E., Simens, M., Sitkin, J., Slama, K., Sohl, I., Sokolowsky, B., Song, Y., Staudacher, N., Such, F. P., Summers, N., Sutskever, I., Tang, J., Tezak, N., Thompson, M. B., Tillet, P., Tootoonchian, A., Tseng, E., Tuggle, P., Turley, N., Tworek, J., Uribe, J. F. C., Vallone, A., Vijayvergiya, A., Voss, C., Wainwright, C., Wang, J. J., Wang, A., Wang, B., Ward, J., Wei, J., Weinmann, C., Welihinda, A., Welinder, P., Weng, J., Weng, L., Wiethoff, M., Willner, D., Winter, C., Wolrich, S., Wong, H., Workman, L., Wu, S., Wu, J., Wu, M., Xiao, K., Xu, T., Yoo, S., Yu, K., Yuan, Q., Zaremba, W., Zellers, R., Zhang, C., Zhang, M., Zhao, S., Zheng, T., Zhuang, J., Zhuk, W., and Zoph, B. Gpt-4 technical report, 2024.
- Patil, S. G., Zhang, T., Wang, X., and Gonzalez, J. E. Gorilla: Large language model connected with massive APIs. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=tBRNC6YemY>.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. Language models are unsupervised multitask learners. *OpenAI*, 2019.
- Stern, M., Shazeer, N., and Uszkoreit, J. Blockwise parallel decoding for deep autoregressive models. In Bengio, S., Wallach, H., Larochelle, H., Grauman, K., Cesa-Bianchi, N., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/file/c4127b9194fe8562c64dc0f5bf2c93bc-Paper.pdf.
- Sun, H., Chen, Z., Yang, X., Tian, Y., and Chen, B. Tri-force: Lossless acceleration of long sequence generation with hierarchical speculative decoding. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=HVK6nl3i97>.
- Sun, Z., Suresh, A. T., Ro, J. H., Beirami, A., Jain, H., and Yu, F. Spectr: fast speculative decoding via optimal transport. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS ’23*, Red Hook, NY, USA, 2023. Curran Associates Inc.
- Wang, Y., Ma, X., Zhang, G., Ni, Y., Chandra, A., Guo, S., Ren, W., Arulraj, A., He, X., Jiang, Z., Li, T., Ku, M., Wang, K., Zhuang, A., Fan, R., Yue, X., and Chen, W. Mmlu-pro: a more robust and challenging multi-task language understanding benchmark. In *Proceedings of the 38th International Conference on Neural Information Processing Systems, NIPS ’24*, Red Hook, NY, USA, 2025. Curran Associates Inc. ISBN 9798331314385.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS ’22*, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Zhou, Y., Lyu, K., Rawat, A. S., Menon, A. K., Rostamizadeh, A., Kumar, S., Kagy, J.-F., and Agarwal, R. Distillspec: Improving speculative decoding via knowledge distillation. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=rsY6J3ZaTF>.

A. Proof of Uncertainty Reduction Using Bidirectional Context

In this section, we provide a formal, information-theoretic proof that incorporating bidirectional context strictly reduces the uncertainty of selecting acceptable tokens.

Theorem 1. *When the suffix $\mathbf{x}_{>t}$ provides additional information about a token X_t given the prefix $\mathbf{x}_{<t}$ (i.e., the conditional mutual information $I(X_t; \mathbf{x}_{>t} | \mathbf{x}_{<t}) > 0$), the uncertainty about X_t , measured by conditional entropy, is strictly reduced when using the bidirectional context $\mathbf{c}_t = (\mathbf{x}_{<t}, \mathbf{x}_{>t})$.*

$$H(X_t | \mathbf{c}_t) < H(X_t | \mathbf{x}_{<t}) \quad (9)$$

Proof. The proof follows directly from the definition of conditional mutual information.

1. The definition of conditional mutual information between two random variables A and B given a third variable C is:

$$I(A; B | C) = H(A | C) - H(A | B, C) \quad (10)$$

2. We substitute our variables: $A = X_t$, $B = \mathbf{x}_{>t}$, and $C = \mathbf{x}_{<t}$.

$$I(X_t; \mathbf{x}_{>t} | \mathbf{x}_{<t}) = H(X_t | \mathbf{x}_{<t}) - H(X_t | \mathbf{x}_{>t}, \mathbf{x}_{<t}) \quad (11)$$

3. By our definition, the bidirectional context is $\mathbf{c}_t = (\mathbf{x}_{<t}, \mathbf{x}_{>t})$. Therefore, the final term is equivalent to the entropy conditioned on the bidirectional context:

$$H(X_t | \mathbf{x}_{>t}, \mathbf{x}_{<t}) = H(X_t | \mathbf{c}_t) \quad (12)$$

4. The premise of our theorem states that $I(X_t; \mathbf{x}_{>t} | \mathbf{x}_{<t}) > 0$. Substituting this into the equation from step 2 gives:

$$H(X_t | \mathbf{x}_{<t}) - H(X_t | \mathbf{c}_t) > 0 \quad (13)$$

5. Rearranging the terms yields the final result:

$$H(X_t | \mathbf{c}_t) < H(X_t | \mathbf{x}_{<t}) \quad (14)$$

This concludes the proof, demonstrating that conditioning on bidirectional context logically leads to lower uncertainty.

A.1. Discussion on the Premise

It is crucial to understand the role of the premise, $I(X_t; \mathbf{x}_{>t} | \mathbf{x}_{<t}) > 0$. From a Natural Language Processing (NLP), this premise is not merely an assumption but a well-established empirical fact. Language is inherently structured, and for any non-trivial linguistic context, the suffix almost always contains information that helps resolve ambiguity or refine the prediction of a token (Devlin et al., 2019). Therefore, while our proof relies on a mathematical assumption, its real-world applicability is grounded in the fundamental nature of human language.

B. Verifier Training

B.1. Statistics

Table 5 shows the training data of verifiers across judge decoding methods. Following previous works, AutoJudge uses math and coding datasets to train the judge verifier. It assigns a token label as acceptable if the newly generated response after replacing a token preserves the original final answer in math and coding problems. To further utilize general datasets like Dolly15k, we implement AutoJudge₊ by generating token labels using LLMs-as-Judge approaches. Specifically, LLMs decide whether the original response and the alternative response are semantically [SAME] or [DIFFERENT]. We assign a token label as acceptable for tokens that results in [SAME].

Table 5. Data for training verifier of each judge decoding methods.

Target Model	Method	Dataset	Num. of samples	Label Rate
Llama-3.1-8B	AutoJudge	GSM8K (train), LiveCodeBench (train)	14896	13633
	AutoJudge+	GSM8K (train), LiveCodeBench (train), Dolly15k	39069	37261
	SelfJudge	GSM8K (train), LiveCodeBench (train), Dolly15k	69432	42253
Qwen-2.5-7B	AutoJudge	GSM8K (train), LiveCodeBench (train)	24748	13068
	AutoJudge+	GSM8K (train), LiveCodeBench (train), Dolly15k	55757	44067
	SelfJudge	GSM8K (train), LiveCodeBench (train), Dolly15k	95128	58482

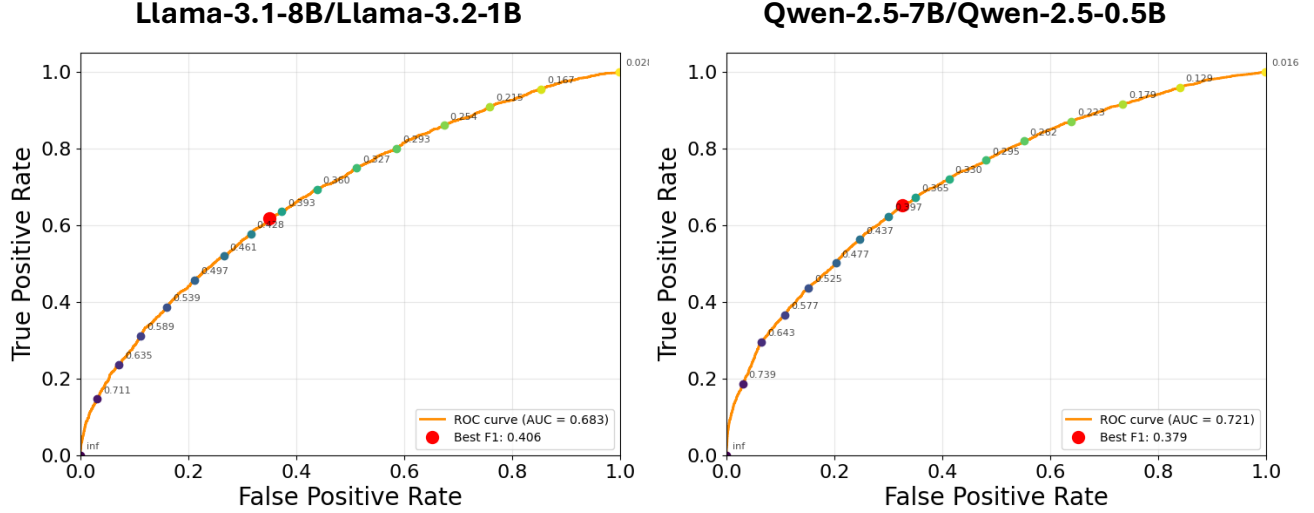


Figure 5. ROC curve of our judge verifier. We train each verifier for Llama-3.1-8B/Llama-3.2-1B and Qwen-2.5-7B/Qwen-2.5-0.5-B, individually. Red point represents threshold value with the best F1 score on validation set.

B.2. Training Logistic Regressor

We employed a logistic regression classifier to address the binary classification problem, which determines acceptable and unacceptable token. We optimized hyperparameters through grid search over the L_2 regularization parameter C . We explored C values from 0.001 to 100 across 10 logarithmically spaced points. Finally, we selected the verifier with the best ROC-AUC.

During inference phase, we apply the trained verifier our two-phase verification method. For the judge verification phase, we select the threshold θ to decide the acceptable tokens. We set the θ to value of the best recall and the best F1 as shown in Figure 5.

B.3. Dataset Generation Time

AutoJudge generates new responses by replacing each mismatched token and completing the text, which is computationally expensive. This requires generating responses $\text{Num. of Queries} \times \text{Num. of Mismatched Tokens}$ times. In contrast, our approach simply computes likelihood differences from the original response without generating new text. We only need to partially prefill a few tokens to calculate these likelihoods. As *partially prefilling* requires much less computational cost compared to *generation*, our approach is significantly efficient, making our method applicable to large models. Using Llama-3.1-8B on NVIDIA V100 GPUs, our SelfJudge method processed 69,432 data points in 8 hours, while AutoJudge processed only 14,896 data points in 82 hours.

C. Experimental Configuration

For mathematical reasoning evaluation, we utilize GSM8K (Cobbe et al., 2021) and MATH-500 (Lightman et al., 2023) using the standard train/test split and evaluation prompts in the benchmark. For code generation evaluation, we leverage LiveCodeBench (Jain et al., 2025). Since this benchmark does not provide predefined train/test splits, we implement a

4-fold cross-validation strategy. For the summarization task, we employ the CNN/DailyMail datasets (Nallapati et al., 2016) using the conventional test split. For general knowledge assessment, we evaluate on MMLU (Hendrycks et al., 2021), a multiple-choice QA benchmark. While the original MMLU benchmark employs a multiple-choice prompt that encourages language models to directly answer among four choices, we apply chain-of-thought prompting (Wang et al., 2025). This modification allows us to simultaneously assess both QA task performance and inference efficiency, taking into account the trade-off between accuracy and the average accepted lengths.

C.1. Math Reasoning Benchmarks

GSM8K (Cobbe et al., 2021) is a dataset of high-quality, diverse grade-school math word problems. We used 1,319 test problems for our evaluation. The MATH-500 dataset contains a subset of 500 problems from the MATH benchmark, which was created by (Lightman et al., 2023). For our evaluation, we used a prompt that required the answer format to appear between [answer] and [\answer]. After extracting the answers, we normalized the format by removing uppercase letters and certain operators. We finally check the correctness of the answer with ground truths to measure accuracy.

C.2. Coding Benchmark

LiveCodeBench (Jain et al., 2025) is a continuously updating coding benchmark for evaluating Large Language Models. It evaluates LLMs across multiple capabilities including code generation, self-repair, test output prediction, and code execution. We used release version 0.0.5, which contains 880 programming problems, and applied 4-fold cross-validation to create training and test datasets. We evaluated performance using the Pass@1 metric, which measures the percentage of problems successfully solved on the first attempt when evaluated against comprehensive test suites.

C.3. Summarization Benchmark

The CNN/DailyMail dataset (Nallapati et al., 2016) is an English-language dataset containing over 300,000 unique news articles written by journalists at CNN and the Daily Mail. We evaluated summarization quality on 11,490 test samples from version 2.0.0, which supports both extractive and abstractive summarization. Since our evaluation focuses on relaxed speculative decoding methods that maintain token-level equivalence with standard decoding, traditional metrics like ROUGE cannot effectively differentiate between methods—all produce identical outputs in terms of content. Therefore, we instead report factual consistency scores to assess summarization quality (Laban et al., 2022). We measure factual consistency using a Natural Language Inference (NLI) model to classify the relationship between source articles and generated summaries. Specifically, we identify sentences that are either entailed by or contradict the source material. The factual consistency score (FC) is computed as:

$$\text{Factual Consistency} = \frac{1}{M} \sum_{i=1}^M (\text{Entailments}_i - \text{Contradictions}_i) \quad (15)$$

where M is the number of summary sentences evaluated. We employed the `deberta-large-mnli` model for NLI classification due to its strong performance on factual verification tasks. In Table 7, we can observe the result of summarization task in more details. As we mentioned, ROUGE scores, which depend on token-level alignment, do not show distinctive difference across methods. Hence, we selected the FC score as our summarization task metric, as it reflect more semantic information by considering entailment of fact when evaluating results.

C.4. Multiple Choices Benchmark

MMLU (Hendrycks et al., 2021) is a massive multitask benchmark containing 14,042 multiple-choice questions across 57 tasks. The benchmark spans diverse domains including humanities, social sciences, hard sciences, and other fields, covering subjects from elementary mathematics to US history, computer science, and law. While the original MMLU evaluation prompts models to directly select answers from 'A', 'B', 'C', or 'D', we adopted the chain-of-thought prompting approach from (Wang et al., 2025) to measure both inference efficiency and task performance simultaneously.

Table 6. Coding Task Performance comparison on Llama-3.1-8B/Llama-3.2-1B.

Method	m / Pass@1	Δ_{task}
SD	7.88 / 8.6	0.0
Top- k	13.36 / 0.7	-7.9
AutoJudge-R	15.78 / 0.7	-7.9
AutoJudge-F	19.63 / 2.1	-6.5
SelfJudge-R	8.12 / 9.7	+1.1
SelfJudge-F	9.52 / 10.0	+1.4

Table 7. Summarization Task Performance Comparison on Llama-3.1-8B/Llama-3.2-1B across various SD methods. m is the averaged accepted length.

Method	m	ROUGE1	ROUGE2	ROUGE-L	FC
SD	4.35	27.1	10.8	22.4	64.6
TopK	7.49	26.8	10.5	22.0	62.5
AutoJudge-R	4.36	27.0	10.8	22.4	64.5
AutoJudge-F	4.74	27.1	10.9	22.5	64.5
SelfJudge-R	4.92	27.5	11.0	22.8	65.5
SelfJudge-F	6.05	26.2	10.3	21.5	63.9

Table 8. Data generation time comparison on Llama-3.1-70B/8B.

Method	53K labels	1M labels
AutoJudge	120 hours	~100 days
SelfJudge	8.5 hours	~6 days

D. Additional Experiments

D.1. Scaling to Large Model.

Data Generation Cost and Scalability. To examine the scalability of SelfJudge to larger models, we conducted additional experiments with Llama-3.1-70B/8B as target and draft models, respectively. Following the same protocol as our main experiments, we sampled 500 instances from GSM8K (train), 220 from Live Code Bench (train), and 500 from Dolly15k. SelfJudge generated a total of 53,318 token labels for verifier training.

Table 8 presents the data generation time comparison between SelfJudge and AutoJudge. Using four H100 GPUs, SelfJudge requires only 8.5 hours to generate these labels, while AutoJudge required 120 hours for the same dataset—achieving a 14 $\times$ speedup. It is attributed to the fact that our data generation only requires the prefill operation of the target model’s response given a mismatched index, whereas AutoJudge needs to complete the response generation to obtain the final answer.

While our results demonstrate that using 53,318 token labels as training data is sufficient for effective verifier, scenarios requiring higher-capacity verifiers may necessitate larger datasets. SelfJudge would require approximately 6 days to generate 1 million token labels, which remains manageable, whereas AutoJudge would require approximately 100 days, making data generation prohibitively expensive.

Scaling to a larger models using Llama-3.1-70B/8B. We evaluated SelfJudge by measuring both the accepted length (m) and task performance. Due to the prohibitive data generation time, AutoJudge could not complete the data generation within the experimental timeframe and is marked as Out of Time (OOT). Table 2 presents the results on GSM8K and MMLU benchmarks. The result confirms that SelfJudge maintains accuracy while improving accepted length and inference speed compared to SD, even on the 70B target model. This demonstrates that our approach scales effectively to larger models.

D.2. Analysis on Two-stage Verification

Following (Bachmann et al., 2025), we adopt the two-stage paradigm, which incorporates alignment-based and judge-based verification. A token accepted by either method is retained in the final output. This design reflects the fundamental role of judge verification: to recover semantically acceptable tokens that alignment-based methods would conservatively reject.

Ablation Study on Verification Phase.

Table 9 shows that Judge-only verification yields a very low accepted length compared to SD. This is because current judge verifiers are designed as lightweight classifiers optimized to complement alignment-based methods by recovering acceptable tokens that alignment would miss. In fact, both (Bachmann et al., 2025) and we focus on setting the threshold of the verifier (classifier) to accurately classify tokens that must be rejected. On the other hand, we allow for cases where some acceptable tokens are incorrectly rejected, since alignment-based verification can correct these errors without any additional

Table 10. Frequently appeared token acceptance by SelfJudge on Math Problem (GSM8K)

Category	Origin	Alter.	Count
Similar Semantic	solve	find	22
	calculate	find	7
	x	*	4
	find	calculate	4
	number	amount	2
Punctuation/ Format	amount	charge	2
	.\n\n	.	9
	.	.	6
	.	.\n\n	6
	:	:\n	5
	(	+	4
	To	\n	4
	\n\n	So	3
	,	.	3
	per	.\n\n	3
	:\n\n	:\n	2
	(	.	2
	)	.	2
	.\n\n	.\n	2
	()	(	2
	()	the	2
	\n\n	Since	2
	:\n	:\n	2
	:	:\n\n	2
Preposition/ Article	in	per	6
	/	per	2
	/p	per	2
	per	of	2
	a	one	2
Sentence Starter	.	and	2
	Let	To	5
	1	We	4
	Next	Now	3
	The	Since	2
	let	we	2

Table 11. Frequently appeared token acceptance by SelfJudge on Coding Problem (LiveCodeBench)

Category	Origin	Alter.	Count
Line Break/ Indentation Changes	\n	.\n\n	44
	:\n	:\n\n	21
	(8 spaces)\n	(7 spaces)	7
	(7 spaces)	(8 spaces)\n	5
	);\n\n	);\n	4
	))\n\n	))\n	4
	of	\n	4
	\n	.\n(8 spaces)\n	3
	to	\n	3
	\n\n	\n	3
	\n\n	\n\n	2
	\n	\n	2
	.\n	.\n\n	2
	for	\n	2
	);\n	);\n\n	2
	()	()\n\n	2
	in	\n	2
	\n	of	2
	\n\n	\n	2
	\n\n	\n\n	2
Punctuation/ Brackets Changes	()\n	s	6
	:	:\n\n	4
	.	.	3
Synonyms/ Similar Meaning	==	!=	3
	code	Python	5
	#	total	4
	number	length	3
	number	element	3
	return	find	2
	max	maximum	2
	count	number	2
	number	count	2

Table 12. Frequently appeared token acceptance by SelfJudge on General Reasoning (MMLU)

Category	Origin	Alter.	Count
Punctuation/ Format Changes	.	.	7
	.	.	7
	.	.\n\n	6
	.\n\n	.	5
	.	)(	2
	because	.\n\n	2
	\n\n	\n	2
	,	a	2
	(	.	2
	.	.\n\n	2
	.\n\n	\n	2
	\n\n	In	2
	)	)	2
	,	.	2
	;\n\n	;\n	2
Articles/ Determiners Changes	a	the	5
	the	a	5
	the	this	3
	the	its	2
	a	an	2
	the	an	2
	we	the	2
	its	the	2
	known	a	2
	a	every	2
Mathematical/ Logic Terms Changes	in	a	2
	validity	correct	6
	Statement	statement	5
	correctness	correct	3
Verbs/ Predicates Changes	answer	correct	3
	let	we	7
	satisfy	make	3
	must	is	3
	use	multiply	2
	is	does	2
	is	means	2

cost. Therefore, Judge-only achieves the low performance as standalone, while it improves the overall performance when it is combined with the alignment-based verification as shown in SelfJudge.

Table 9. Ablation study on the two-stage verification phase on Llama-3.1-8B/Llama-3.2-1B.

Method	GSM8K	LiveCodeBench	MMLU
Alignment-only(SD)	9.14 / 80.7	7.88 / 8.6	4.36 / 65.0
Judge-only	2.08 / 80.2	1.76 / 9.2	1.45 / 65.0
Two-stage (SelfJudge)	11.29 / 80.5	9.52 / 10.0	6.38 / 62.7

D.3. Qualitative Analysis

Tables 10, 11, and 12 present the most frequently accepted tokens by SelfJudge while they are rejected by Standard SD across three distinct domains: mathematical reasoning (GSM8K), coding (LiveCodeBench), and general reasoning (MMLU). These results reveal several key insights about the flexible nature of our approach and domain-specific characteristics of semantically preserving token modifications that do not compromise response quality. Domain-Agnostic Strengths. Across all three tasks, SelfJudge demonstrates consistent acceptance of tokens that preserve semantic meaning and logical equivalence. Mathematical terms like “solve $\rightarrow$ find” and calculate $\rightarrow$ find” in GSM8K, along with synonymous replacements such as number $\rightarrow$ element” in coding tasks, highlight the model’s ability to identify semantically equivalent tokens that would not alter the fundamental meaning of the response. Moreover, SelfJudge successfully allows minor formatting discrepancies such as “ $\rightarrow$ ” and “ $\rightarrow$ ” and “ $\rightarrow$ ” and “ $\rightarrow$ ”. This flexibility enables speculative decoding to achieve significant latency reduction while preserving the semantic integrity and quality of the original target model response.

Mathematical Reasoning Characteristics. In GSM8K, SelfJudge exhibits a particular pattern of accepting mathematical vocabulary variations that preserve computational meaning. The frequent acceptance of transformations like $x \rightarrow *$ and find $\rightarrow$ calculate” indicates that SelfJudge correctly identifies when mathematical operations remain functionally equivalent. Additionally, the acceptance of sentence starters like “Let $\rightarrow$ To” and “Next $\rightarrow$ Now” demonstrates that these

discourse markers do not fundamentally alter the mathematical reasoning process, allowing for flexible problem-solving exposition styles without compromising the logical flow.

Coding Domain Specificity. LiveCodeBench results reveal how SelfJudge achieves high acceptance rates while maintaining code functionality. SelfJudge correctly accepts functionally equivalent code transformations, such as whitespace normalization (“ $\backslash n \rightarrow \backslash n \backslash n$ ”) and formatting changes that do not affect program execution. Notably, synonym acceptance for programming concepts (“max $\rightarrow$ maximum”, “count $\rightarrow$ number”) shows the model’s ability to recognize coding-specific terminology variations that preserve semantics. The acceptance of structural changes in brackets and punctuation effectively improve the efficiency of judge verification while preserving the target model’s intention.

General Reasoning Adaptability. Table 5 exhibits frequent acceptance of transformations involving punctuation and formatting variations, indicating that such changes do not alter the semantic content or factual accuracy of responses. Particularly noteworthy is the acceptance of articles/determiners changes (“a $\rightarrow$ the”, “the $\rightarrow$ a”) and logical term modifications (“validity $\rightarrow$ correct”), which suggests SelfJudge can distinguish between semantically equivalent expressions that convey identical meaning. We argue that minor linguistic variations with these token changes do not compromise the informational content or reasoning quality of the original response.

D.4. Hyperparameter Sensitivity Analysis

Prediction Threshold Analysis. To comprehensively analyze the relationship between inference efficiency and task performance across varying threshold values, we conduct a systematic investigation into the threshold sensitivity of each method with respect to parameter θ . In Figure 3, our experimental results demonstrate that SelfJudge consistently achieves superior performance compared to all baseline methods across diverse thresholds. This robust performance advantage indicates that SelfJudge represents a fundamentally superior approach, delivering optimal trade-offs between accepted sequence lengths and task-specific performance metrics while maintaining stability across different threshold settings. It implies that our method’s effectiveness is not contingent upon fine-tuned hyperparameter selection, thereby enhancing its practical applicability and robustness in real-world deployment scenarios.